

Writing Plan: High-Performance SQMC on GPU

Thesis / narrative arc

The chapter tells one coherent story:

SQMC is the right algorithm for high-performance filtering, and the GPU is the right hardware for SQMC — because both are built on the same primitives: low-discrepancy point generation, Hilbert-curve ordering, and massively parallel bit-level operations.

- **Papers 1–3** give the *algorithm* (SQMC + its theory).
- **Papers 8–9** give the *engineering evidence* that the primitives SQMC needs are exactly the ones that run fast on GPUs.
- **Our contribution** is the bridge: implementing SQMC’s resampling (Hilbert sort) and propagation on the GPU, and measuring the payoff.

Section A — Algorithmic Foundations: SMC, QMC, and SQMC

Draft

We consider a state-space model with latent Markov process X_t and observations Y_t , and target the filtering distribution $p(x_t | y_{1:t})$. Sequential Monte Carlo (SMC) approximates this distribution with a weighted particle set $\{x_t^n, w_t^n\}_{n=1}^N$ propagated by a two-step recursion: a *prediction* step that samples new particles from a proposal, and a *correction* step that reweights them by the likelihood. Because the weights degenerate over time, a *resampling* step duplicates high-weight particles and discards low-weight ones. The SMC estimator converges at the Monte Carlo rate $O_P(N^{-1/2})$.

Quasi-Monte Carlo (QMC) replaces random draws with deterministic low-discrepancy point sets. By the Koksma–Hlawka inequality, the integration error is bounded by the product of the function’s total variation and the point set’s star discrepancy, which for well-constructed sequences is $O(N^{-1}(\log N)^s)$ — far better than the MC rate for smooth integrands.

Sequential quasi-Monte Carlo (SQMC) combines these ideas. Instead of sampling the proposal stochastically, each particle is propagated deterministically through a low-discrepancy point set,

$$x_t^n = \Gamma_t(x_{t-1}^{a_t^n}, u_t^n), \quad u_t^n \in \mathcal{U}[0, 1]^d,$$

where a_t^n is the ancestor index from resampling and u_t^n are QMC points. Resampling is performed *before* propagation: particles are sorted along a Hilbert curve, then an inverse-CDF transform is applied to the first coordinate of the QMC points. The complexity is $O(N \log N)$ per step, and the error rate is smaller than the Monte Carlo rate $O_P(N^{-1/2})$.

Beyond its faster convergence rate, SQMC also offers a stronger *safety* guarantee than SMC. Whereas a particle filter’s error can, with probability one, exceed any threshold $\kappa \in (0, 1/2)$ at infinitely many time instants regardless of N , SQMC’s error vanishes uniformly in time: $\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| = 0$ almost surely. This uniform-in-time control makes SQMC especially attractive for long-horizon filtering.

The curse of dimensionality in SQMC

SQMC’s advantage over SMC rests on the low discrepancy of the QMC point set $\{u_t^n\}_{n=1}^N$ in $[0, 1]^d$. By the Koksma–Hlawka inequality, the integration error is bounded by the product of the integrand’s total variation and the star discrepancy $D_d^*(\{u_t^n\})$, which for a (t, s) -sequence satisfies

$$D_d^*(\{u_t^n\}) \leq C_{d,t,b} \frac{\log(N)^d}{N}.$$

The $\log(N)^d$ factor is the source of the **curse of dimensionality**: as the state dimension d grows, the discrepancy bound degrades, and the QMC advantage over the MC rate $O_P(N^{-1/2})$ shrinks. In high dimension, SQMC’s error rate approaches that of SMC, and the low-discrepancy structure is effectively lost.

This is the central limitation of SQMC identified by and : SQMC performs well only when the effective dimension of the filtering problem is small. The remedy, developed in Chapter~2, is to **reduce the dimension** of the problem before applying SQMC — for example by Rao-Blackwellization, which integrates out a subset of the latent variables analytically and runs SQMC on the smaller remaining subset. This is precisely the motivation for the football application.

Key points to convey

- SMC: prediction/correction/resampling recursion; weight degeneracy; $O_P(N^{-1/2})$ rate.
- QMC: low-discrepancy sequences; Koksma–Hlawka; $O(N^{-1})$ for smooth integrands.
- SQMC: deterministic propagation Γ_t ; Hilbert-sorted resampling; $O(N \log N)$; better-than-MC rate.
- Safety: SQMC has uniform-in-time error control; PF does not .
- Curse of dimensionality: discrepancy bound $O(\log(N)^d/N)$ degrades with d ; motivates dimension reduction (Rao-Blackwellization).

Section B — Why SQMC Maps Naturally onto Accelerators

Draft

The primitives that SQMC needs are exactly the ones that run efficiently on parallel hardware (GPUs, TPUs, and multi-core CPUs). Low-discrepancy point generation is built from bit-level operations — Gray codes, radical inverses, and digital nets — that are embarrassingly parallel and map directly onto accelerator threads . Scrambling (e.g., Owen scrambling) recovers unbiased error estimates while preserving low discrepancy, and is likewise a per-thread operation. Space-filling curves, and in particular the Hilbert curve, provide a locality-preserving ordering that is computed from bit interleaving and reflection — again a purely parallel, deterministic computation.

This is not merely a theoretical observation. In production rendering, enumerate a low-discrepancy sequence along a Hilbert curve superimposed on the pixel raster, achieving noise characteristics desirable for the human visual system at very low sampling rates. Their algorithms are deterministic, require neither randomization nor costly optimization nor lookup tables, and are demonstrated in a massively parallel light transport system. This is direct evidence that the Hilbert-ordering primitive — the same one SQMC uses for resampling — scales to production GPU workloads.

Key points to convey

- QMC primitives (Gray codes, radical inverse, scrambling) are bit-level and embarrassingly parallel.
- The Hilbert curve gives a locality-preserving, deterministic, parallel ordering.
- Paper 8 is production-validated proof that the Hilbert primitive scales on GPUs.

Section C — A JAX Implementation of SQMC

Draft

We implement the SQMC pipeline in JAX, which compiles to accelerator kernels (GPU, TPU, or CPU) and lets us switch backends with a single configuration flag. Two components are central.

Hilbert sort. Our `hilbert_sort.py` computes a generalized d -dimensional Hilbert index for arbitrary dimension $2 \leq d \leq 62$, packed into a single 62-bit integer. The implementation follows the recursive orientation construction: coordinate bits are transposed into per-level chunks, each chunk is decoded through a Gray-code travel step that tracks the start/end corners of the current sub-cube, and the resulting per-level indices

are packed into one integer. Sorting the particles by this index yields the locality-preserving order required by SQMC’s resampling step.

QMC point generation. Our `qmc.py` provides Halton and Sobol sequences with optional scrambling. The Halton sequence uses the radical inverse in a distinct prime base per dimension, with Owen-style digit permutations for scrambling. The Sobol sequence uses Joe–Kuo direction numbers with a left linear matrix scramble plus digital shift. Both are vectorized over the particle array, so generating N points in d dimensions is a single batched GPU operation.

Together these provide the two primitives of the SQMC loop: low-discrepancy point generation (propagation) and Hilbert-sorted resampling. The full loop — generate points, propagate particles, compute weights, Hilbert-sort, resample — runs entirely on the accelerator.

Key points to convey

- JAX compiles to accelerator kernels; backend switch (GPU/TPU/CPU) is a config flag.
- Hilbert sort: generalized d -D index, packed 62-bit, recursive orientation construction.
- QMC generation: Halton/Sobol, radical inverse, Owen scrambling, vectorized over particles.
- The full SQMC loop runs on the GPU.

Section D — Empirical Evaluation: Throughput and Safety

Draft

Our focus is GPU vs CPU throughput — the time-to-solution. Statistical convergence and dimension scaling are already established by the papers; the novel contribution is *how fast* the GPU reaches a given accuracy. We fix a filtering task and, for each method (SMC-CPU, SQMC-CPU, SQMC-GPU), measure the filtering error as a function of wall-clock run time. Because JAX lets us toggle the backend, the same code runs on both CPU and GPU, isolating the hardware effect from any implementation differences.

We report four quantities. First, the headline *progress* plot: filtering error against run time, showing that SQMC-GPU reaches a target error far sooner than the alternatives. Second, throughput in particles per second against N , demonstrating that GPU throughput stays high as N grows while CPU throughput flattens. Third, the speedup ratio $\text{time}_{\text{CPU}}/\text{time}_{\text{GPU}}$ against N , which should rise with N as the GPU’s parallelism is better utilized. Fourth, a per-step time breakdown, showing that the Hilbert sort and point generation — the bit-level, embarrassingly parallel steps — are where the GPU wins most.

In addition to these throughput plots, we include a *long-horizon safety* experiment that empirically illustrates the results of . On the toy linear-Gaussian model of that paper, we track the Kolmogorov distance $\|\hat{\eta}_t^N - \hat{\eta}_t\|$ over a long horizon for both PF and SQMC. The PF error occasionally spikes above any threshold (its “infinitely often” failure), while the SQMC error stays bounded and shrinks with N (uniform-in-time control). This gives a *statistical* reason — not just speed — to prefer SQMC on GPU, and motivates the long-horizon framing of the throughput plots.

Key points to convey

- Focus is time-to-solution, not statistical convergence (already in the papers).
- Same JAX code on CPU and GPU isolates the hardware effect.
- Four plots: progress vs. time, throughput vs. N , speedup vs. N , per-step breakdown.
- Expect Hilbert sort and point generation to be the biggest GPU wins.
- Fifth experiment: long-horizon safety, empirically illustrating .

Performance plots (Section D)

Core plot: Progress vs. Run Time (headline figure)

Fix a filtering task; for each method (SMC-CPU, SQMC-CPU, SQMC-GPU), measure how much work gets done as a function of wall-clock time.

- **Y-axis options** (pick the one that tells the story best):
 1. Number of particles processed N (throughput) — simplest, pure speed.
 2. Effective sample size / accuracy achieved — ties to quality.
 3. Filtering error (RMSE) — the “time-to-accuracy” view.
- **X-axis:** wall-clock run time (seconds, log scale).
- **Expected result:** SQMC-GPU’s curve rises much faster (steeper) than SMC-CPU and SQMC-CPU. If error is on the y-axis, the GPU curve drops to a given error level much sooner.

Supporting plots

1. **Throughput scaling: Particles/sec vs. N**
 - X: N (log). Y: particles/sec.
 - Shows GPU throughput stays high (or grows) as N grows, while CPU flattens — the parallel-scaling advantage.
2. **Speedup factor vs. N**
 - X: N (log). Y: $\text{time}_{\text{CPU}}/\text{time}_{\text{GPU}}$.
 - Shows a rising curve — speedup *grows* with N because GPU parallelism is better utilized at larger particle counts. The single most persuasive number.
3. **Per-step breakdown (stacked bar or line)**
 - X: SQMC steps (point generation, Hilbert sort, resampling, propagation, weight update). Y: time per step (or % of total).
 - Shows *where* the GPU wins. Expect Hilbert sort and point generation (bit-level, embarrassingly parallel) to be the biggest wins; propagation may be the bottleneck if not vectorized. Directly supports the “Hilbert sort on GPU” focus.
4. **Time-to-target-error (the “progress” plot done right)**
 - X: run time (log). Y: filtering RMSE (log). Overlay a horizontal dashed line at a target error ϵ .
 - Shows the *time* at which each method crosses the target. SQMC-GPU should cross first.

Suggested figure set (4 figures)

#	Plot	Purpose
1	Progress (error) vs. run time	Headline: time-to-accuracy
2	Throughput (particles/sec) vs. N	Parallel scaling
3	Speedup ratio vs. N	The persuasive number
4	Per-step time breakdown	Where the GPU wins (Hilbert sort)

Fifth experiment: Long-horizon safety (Gerber 2025)

This is a **statistical** experiment (not a throughput plot) that complements the four above. It empirically illustrates the safety results of , which were *proven* but **not simulated** in that paper — so this is a novel empirical contribution.

Setup. Use the toy model from the paper: a 1-D, time-homogeneous linear Gaussian SSM

$$Y_t = X_t + c^{-1/2}Z_t, \quad X_{t+1} = \rho X_t + \sigma W_{t+1},$$

with observations fixed at $y_t = 0$ for all $t \geq 1$. Choose parameters (e.g., $\rho = 0.9$, $\sigma = 1$, $c = 1$). Run the bootstrap PF (multinomial resampling) and SQMC (scrambled $(0, 2)$ -sequence in base 2, as the paper specifies) over a long horizon T (e.g., 10^4 – 10^5).

Metric. Track the Kolmogorov distance $\|\hat{\eta}_t^N - \hat{\eta}_t\|$ over time for several N .

What to show. - **PF:** the error occasionally spikes to $\geq \kappa$ even for large N — demonstrating the “infinitely often” failure ($\mathbb{P}(\sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| \geq \kappa) = 1$). - **SQMC:** the error stays bounded and shrinks as N grows — demonstrating uniform-in-time control ($\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| = 0$ a.s.).

Plot. Error vs. time for a few N values, PF vs. SQMC. This is a compelling visual that the paper lacks.

Framing. Present it as “we empirically illustrate the safety results of Gerber (2025), which were proven but not simulated.” It gives a *statistical* reason (not just speed) to prefer SQMC on GPU, and motivates the long-horizon framing of the throughput plots.

Caveat. The safety result is proven for this specific toy model — present it as an illustration, not a universal claim.

Practical notes

- Use **log–log axes** for error/throughput plots — the slopes are the story (SQMC’s $O(N^{-1})$ vs SMC’s $O(N^{-1/2})$).
- **JAX makes CPU/GPU comparison trivial** — set `jax.config.update('jax_platform_name', 'cpu')` vs. letting it pick the GPU (`qmc.py` already has this commented out).
- **Report the hardware** (GPU model, CPU model) in the caption.
- **Warm up the GPU** before timing (JAX compiles on first call); time the *steady-state* loop, not the first JIT compile.

Writing order (drafting sequence)

1. **Section A first** — mostly literature synthesis from papers 1, 3. Much is already drafted in `1_high_performance_sqmc.tex`.
2. **Section B** — synthesize papers 8, 9 into the “why GPU” argument. Short and high-level.
3. **Section C** — describe the code; connect math to implementation (radical inverse $\leftrightarrow$ Halton, Hilbert index $\leftrightarrow$ resampling).
4. **Section D last** — depends on running the experiments; results shape the narrative.

Key writing tips

- **Lead with the “one story” thesis** in the introduction.
 - **Use paper 8 as the “proof of concept” citation** for the Hilbert primitive — it is production-validated.
 - **Use paper 9 for the “why QMC on GPU” framing** — reproducibility, progressive refinement, scrambling.
 - **Use the safety result as a second pillar** — SQMC is not only faster but *safer* (uniform-in-time error control). Weave this into the introduction and Section D to give the chapter a statistical anchor beyond raw speed.
 - **Be explicit about what is ours vs. the literature:** papers 1–3, 8, 9 establish the algorithm and the primitive; our contribution is the *filtering-specific GPU implementation*, the *performance measurement*, and the *empirical illustration of the safety results* (which were proven but not simulated).
-

Safety of particle filters (Gerber 2025, arXiv:2503.21334)

What the paper establishes

This paper studies the **time evolution** of particle-filter estimates — a topic that had received little attention. Its two main results:

1. **Particle filters are not “safe” over time.** For any number of particles N , with probability one,

$$\|\hat{\eta}_t^N - \hat{\eta}_t\| \geq \kappa \quad \text{for infinitely many } t \geq 1,$$

where $\|\cdot\|$ is the Kolmogorov distance and $\kappa \in (0, 1/2)$. That is, no matter how many particles you use, the PF estimate will occasionally be far from the true filtering distribution at some future time.

2. **SQMC is safe.** For the same toy filtering problem, sequential quasi-Monte Carlo (a randomized QMC version of PF) offers strictly stronger guarantees:

$$\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| = 0 \quad \text{with probability one.}$$

So SQMC’s error vanishes *uniformly in time* as $N \rightarrow \infty$, whereas the PF’s error does not.

The precise error-control bounds

The paper works on a 1-D, time-homogeneous linear Gaussian SSM with observations fixed at $y_t = 0$:

$$Y_t = X_t + c^{-1/2} Z_t, \quad X_{t+1} = \rho X_t + \sigma W_{t+1}.$$

It compares the bootstrap PF (multinomial resampling) against SQMC using a scrambled $(0, 2)$ -sequence in base $b = 2$. The error is measured in the Kolmogorov distance $\|\hat{\eta}_t^N - \hat{\eta}_t\|$.

Negative result for PF (Proposition 2). For any $N \geq 1$ and any $\kappa \in (0, 1/2)$, with probability one,

$$\|\hat{\eta}_t^N - \hat{\eta}_t\| \geq \kappa \quad \text{for infinitely many } t \geq 1.$$

The mechanism: at each step, all particles fall outside any fixed interval $[a, b]$ with probability $\geq \varrho^N > 0$; by the second Borel–Cantelli lemma this happens infinitely often.

Finite-horizon probabilistic bound for PF (Theorem 1). There is a constant $\bar{C}_1$ such that, for all $q \in (0, 1)$ and $T \geq 1$, with probability at least $1 - q$,

$$\max \left\{ \sup_{t \in \{1, \dots, T\}} \|\hat{\eta}_t^N - \hat{\eta}_t\|, \sup_{t \in \{1, \dots, T\}} \|\eta_t^N - \eta_t\| \right\} \leq \bar{C}_1 \delta_{N,T,q}^{1/2} \log(1 + \delta_{N,T,q}^{-1/2}),$$

where $\delta_{N,T,q} = N^{-1}(1 + \log(T/q))$. The bound grows with T through $\log(T/q)$; to keep the error below κ with probability $\geq 1 - q$ (Corollary 1), one needs

$$N \geq N_{T,\kappa,q} = \bar{C}_1' \{1 + \log(1 + \kappa^{-1})\}^2 \kappa^{-2} (1 + \log(T/q)).$$

So N must grow like $\log(T)$ to control the error over horizon T . Proposition 3 shows this $\log(T/q)$ dependence is **sharp**.

Time-uniform a.s. bound for SQMC (Theorem 2). There is a constant $\bar{C}_2$ such that, with probability one,

$$\max \left\{ \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\|, \sup_{t \geq 1} \|\eta_t^N - \eta_t\| \right\} \leq \bar{C}_2 \delta_N^{1/2} \log(1 + \delta_N^{-1/2}),$$

where δ_N is the star-discrepancy bound for the scrambled $(0, 2)$ -sequence:

$$\delta_N = \begin{cases} \frac{\log(N) + 3}{2N}, & N \in \{2^k : k \in \mathbb{N}\}, \\ \frac{\log(N)^2 + 11 \log(2) \log(N) + 18 \log(2)^2}{8 \log(2)^2 N}, & \text{otherwise.} \end{cases}$$

This bound has **no T dependence** — it is time-uniform, holding for all $t \geq 1$ simultaneously, and gives the a.s. result $\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| = 0$. Corollary 2 gives the explicit condition: if $\frac{N}{\log(1+N)^2} \geq \bar{C}'_2 \{1 + \log(1 + \kappa^{-1})\}^2 \kappa^{-2}$, then $\mathbb{P}(\sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| \leq \kappa) = 1$.

The contrast in one line.

	PF (Theorem 1)	SQMC (Theorem 2)
Error bound	$\bar{C}_1 \delta_{N,T,q}^{1/2} \log(1 + \delta_{N,T,q}^{-1/2})$, $\delta_{N,T,q} = N^{-1}(1 + \log(T/q))$	$\bar{C}_2 \delta_N^{1/2} \log(1 + \delta_N^{-1/2})$, $\delta_N \approx \log(N)/N$
Time dependence	Yes — grows like $\log(T)$	No — time-uniform
Mode	Probabilistic ($\geq 1 - q$)	Almost sure
As $T \rightarrow \infty$	Fails infinitely often	Holds for all t

The paper notes (Remark 4) that SQMC’s bound is slightly worse in its N -dependence (the $\log(N)$ term in δ_N) than PF’s probabilistic bound, but SQMC’s advantage is that it is **time-uniform and almost sure** — which is exactly the “safety” property.

Why it matters for the chapter

This is a **theoretical argument in favor of SQMC over SMC** that is independent of the GPU/throughput story. It strengthens the case for SQMC on two fronts:

- **Statistical:** SQMC is not just faster (better convergence rate) — it is *safer* (uniform-in-time error control). This complements the convergence-rate results of paper 1.
- **Practical:** uniform-in-time error control means SQMC’s estimates do not degrade unpredictably over long horizons — relevant for long time-series filtering, which is exactly the regime where GPU throughput matters most.

How to incorporate it

Where it fits in the chapter:

- **Section A (Background)** — cite it alongside paper 1 when stating SQMC’s theoretical advantages over SMC. Add one sentence: *“Moreover, unlike SMC, SQMC provides uniform-in-time error control, in the sense that $\lim_{N \rightarrow \infty} \sup_{t \geq 1} \|\hat{\eta}_t^N - \hat{\eta}_t\| = 0$ almost surely.”*
- **Section D (Performance demonstration)** — use it to *motivate* the time-to-solution framing. The safety result justifies why long-horizon filtering is a meaningful benchmark: SQMC’s advantage compounds over time, so the GPU throughput gain is most visible on long runs. This gives the performance section a theoretical anchor beyond raw speed.

Suggested framing sentence for the introduction:

“SQMC is not only faster than SMC — it is also safer: its error vanishes uniformly in time as the number of particles grows, a property that makes it especially attractive for long-horizon filtering, where GPU acceleration delivers the greatest benefit.”

Caveat to note: the safety result is proven for a *toy filtering problem*, not in full generality. In the chapter, present it as evidence/motivation rather than a universal theorem, and be careful not to overclaim.

Paper reference map

#	Paper	Role
1	Chopin & Gerber (2015) — SQMC (JRSS-B)	Core algorithm
2	Chopin & Gerber (2017) — SQMC intro/diffusion	Dimension reduction
3	Chopin & Papaspiliopoulos (2020) — SMC textbook	Background/theory
8	Keller, Wächter & Binder (2022) — Rendering along the Hilbert Curve	Hilbert primitive, GPU proof-of-concept
9	Keller, Wächter & Binder (2013) — QMC for Graphics Software	Why QMC on GPU
—	Gerber (2025) — Safety of particle filters (arXiv:2503.21334)	SQMC uniform-in-time error control